

PORTADA

Calidad de Software

Semana 12 — Integración continua y el pipeline de calidad

Universidad de Antioquía · Ingeniería de Sistemas



REPASO RELÁMPAGO

Lo que ya tenemos de la semana 11

- **Densidad de defectos** compara defectos contra tamaño; **DRE** mide qué tan bien se atraparon antes del release.
- **Cobertura** mide qué se ejecutó, no qué se verificó; **complejidad ciclomática** mide caminos, no si el código es correcto.
- Ninguna métrica reemplaza el juicio humano.



TRANSICIÓN

Hoy: automatizar todo lo anterior

Pruebas unitarias (semana 10), cobertura, análisis estático con SonarCloud (semana 5), métricas (semana 11) — hasta ahora, cada una se ejecutó manualmente, cuando alguien se acordaba. Hoy se conecta todo en un solo proceso que corre **solo**, cada vez que alguien sube un cambio.

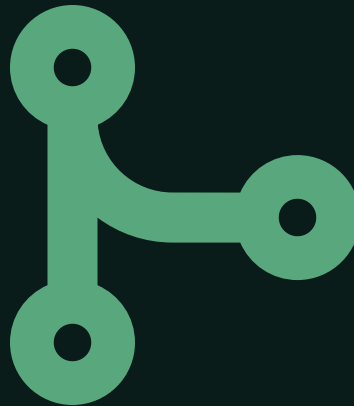


CONCEPTO · ORIGEN

Qué es la integración continua (CI)

Es la práctica de **integrar el código de todo el equipo con mucha frecuencia** —varias veces al día— en una rama compartida, verificando automáticamente en cada integración que todo sigue funcionando.

Nació dentro de **Extreme Programming** (semana 3) y la popularizó **Martin Fowler** en un artículo influyente de 2000, en una época donde era común que los equipos integraran su código cada varias semanas — con resultados dolorosos.



CONCEPTO

Por qué integrar frecuentemente importa

Cuando dos personas modifican el mismo código durante semanas antes de integrarlo, los conflictos que aparecen son grandes, difíciles de resolver, y nadie recuerda ya por qué se tomó cada decisión.

Integrar todos los días —o varias veces al día— hace que cada conflicto sea pequeño, se resuelva rápido, y el costo de encontrar un problema sea mucho menor que dejar que se acumule en silencio.

TRANSICIÓN

Qué hace un pipeline, paso a paso

La verificación automática de la que habla la definición de CI no es un solo paso: es una secuencia de etapas, cada una con su propia responsabilidad.



CONCEPTO

Qué es un pipeline de calidad

Es la secuencia de **etapas automatizadas** que corren, en orden, cada vez que hay un cambio de código: si una etapa falla, las siguientes no se ejecutan y el cambio queda marcado como roto.

El pipeline de este curso tiene cuatro etapas: **build, test, cobertura y análisis estático**. Cada una responde una pregunta distinta sobre el mismo cambio.



CONCEPTO

Etapa 1 — Build

Compila o empaqueta el código. Es la verificación más básica de todas: **¿el código siquiera se puede construir?**

Si el build falla, ninguna etapa posterior tiene sentido — no hay nada que probar todavía. Es, casi siempre, la etapa más rápida y la primera en detectar un error de sintaxis o una dependencia faltante.



CONCEPTO

Etapa 2 — Test

Ejecuta la suite de pruebas automatizadas —empezando por las unitarias de la semana 10, la base de la pirámide de Cohn. **¿El código hace lo que se espera que haga?**

Como estas pruebas corren en cada cambio, funcionan también como una red de **pruebas de regresión** constante (semana 7): cualquier cosa que se rompa por un cambio nuevo se detecta aquí, casi de inmediato.



CONCEPTO

Etapas 3 — Cobertura

Genera y **publica** el reporte de cobertura (semanas 5, 8 y 11) del código que se acaba de probar. No prueba nada por sí sola — mide qué tanto del código ejercitaron las pruebas de la etapa anterior.

Publicarlo automáticamente, en cada cambio, es lo que permite ver la tendencia en el tiempo — no solo una foto aislada.



CONCEPTO

Etapas 4 — Análisis estático

Corre una herramienta como **SonarCloud** (semana 5) sobre el código, sin ejecutarlo, buscando bugs potenciales, vulnerabilidades y code smells. **¿El código en sí tiene señales de problemas, más allá de si las pruebas pasan?**

Es la última etapa porque suele ser la más lenta, y porque no tiene sentido analizar a fondo un código que ni siquiera compila o pasa sus pruebas.



Las cuatro etapas, en orden

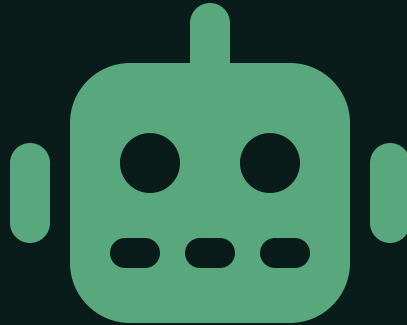
Etapas	Pregunta que responde
1. Build	¿El código se puede construir?
2. Test	¿Hace lo que se espera?
3. Cobertura	¿Qué tanto se probó?
4. Análisis estático	¿Hay señales de problemas en el código en sí?

Si cualquier etapa falla, el pipeline se detiene ahí — las siguientes ni se ejecutan.

TRANSICIÓN

Quién ejecuta todo esto, automáticamente

Nadie corre estas cuatro etapas a mano cada vez. Hace falta una herramienta que las dispare sola, cada vez que alguien sube un cambio.



CONCEPTO · HERRAMIENTA

Qué es GitHub Actions

Es el sistema de automatización integrado en GitHub —gratuito para repositorios públicos, la herramienta que usa este curso. Se configura con tres conceptos:

- **Workflow:** el pipeline completo, definido en un archivo YAML dentro de `.github/workflows/`.
- **Job:** un conjunto de pasos que corre en una máquina — un workflow puede tener varios.
- **Trigger:** el evento que dispara el workflow — típicamente `on: push` o `on: pull_request`.



EJEMPLO

Un workflow mínimo

```
name: Pipeline de calidad

on: [push, pull_request]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build
        run: mvn compile
      - name: Test con cobertura
        run: mvn test jacoco:report
```

Cuatro pasos, dos triggers: cada `push` o cada `pull_request` ejecuta el build y las pruebas automáticamente, sin que nadie tenga que acordarse.

CONCEPTO · RECORDATORIO

El Quality Gate, dentro del pipeline

Semana 5: el Quality Gate son las condiciones —cero bugs nuevos, cobertura mínima— que un análisis debe cumplir para considerarse aprobado. Dentro del pipeline, ese Quality Gate se vuelve **un paso más**: si no se cumple, el pipeline completo queda en rojo.



CONCEPTO

El pipeline como puerta, no como sugerencia

La configuración más común —y la más útil— es que un **pull request no se pueda fusionar** a la rama principal mientras el pipeline esté en rojo. No es una recomendación que el equipo puede ignorar: es un bloqueo técnico.

Esto convierte al pipeline en el corrector automático que libera tiempo del docente —o del líder técnico— para revisar criterio y diseño, en vez de verificar a mano que las pruebas pasen.



APLICADO

De vuelta a Knight Capital

Un pipeline de CI con estas cuatro etapas no habría dejado pasar el defecto de 2005: al mover la función de cantidad acumulada, la etapa de **test** habría ejecutado la suite de regresión sobre *Power Peg* y habría fallado de inmediato.

Y el despliegue manual e inconsistente en los ocho servidores —el ingrediente final del desastre— es exactamente el tipo de proceso que un pipeline automatizado elimina: todos los servidores reciben el mismo código, de la misma forma, cada vez.



INTERACCIÓN

Ejercicio rápido: ¿en qué etapa falla?

5 minutos:

1. Un archivo tiene un error de sintaxis.
2. El código compila, pero una prueba unitaria existente falla.
3. Todo pasa, pero SonarCloud reporta una vulnerabilidad de seguridad crítica.

¿En cuál de las cuatro etapas se detiene el pipeline en cada caso?

Este ejercicio es el calentamiento del taller de hoy.

CONCEPTO EN DISPUTA

¿Un pipeline verde garantiza que se puede desplegar con confianza?

Un pipeline en verde significa que el código compila, pasa las pruebas escritas, tiene la cobertura mínima y no tiene hallazgos críticos de SonarCloud. Es tentador leer eso como "está listo para producción".

Pero el pipeline solo verifica lo que alguien decidió automatizar. Si las pruebas no cubren un caso real de uso, o si nadie escribió una prueba de rendimiento, el pipeline puede estar perfectamente verde sobre un sistema con un problema serio. **El pipeline automatiza la verificación existente — no genera verificación que nadie pensó en escribir.**



SÍNTESIS

Lo que hay que llevarse de hoy

1. La **integración continua** integra código con frecuencia, verificando automáticamente en cada integración.
2. Un **pipeline de calidad** encadena build, test, cobertura y análisis estático — si una etapa falla, las siguientes no corren.
3. **GitHub Actions** ejecuta el pipeline automáticamente ante eventos como `push` o `pull_request`.
4. El **Quality Gate** convierte al pipeline en una puerta real: bloquea el merge, no solo lo sugiere.
5. Un pipeline verde es **necesario, no suficiente** — solo verifica lo que alguien pensó en automatizar.



INSTRUCCIONES

Taller de hoy: pipeline de CI operativo

2 horas, en sala de Zoom asignada — equipo fijo de Fábrica-Escuela

Sobre el repositorio de práctica del equipo, configuren un pipeline que:

- Se dispare automáticamente en cada `push` o `pull request`.
- **Ejecute la suite** de pruebas automatizadas del proyecto.
- **Publique el reporte de cobertura** como parte del resultado del workflow.
- Entregable: el archivo `.github/workflows/*.yaml` commiteado en el repositorio, más un enlace a una ejecución en verde del workflow (pestaña *Actions* de GitHub).

Herramienta: [GitHub Actions](#) · **La entrega cierra al terminar las dos horas.** Recuerden: al enviar, cada integrante reporta en privado quién trabajó.



CIERRE

Para la próxima semana

Antes de la clase de la semana 13, hay que llegar con esto leído:

- Frameworks de automatización: Selenium, Playwright, Cypress, REST Assured. Selección según contexto.

Seguimos con los equipos fijos de **Fábrica-Escuela**. El taller de la semana 13: primer test end-to-end automatizado sobre The Internet (Herokuapp), con evidencia de ejecución.

